

Voici le DAT généré selon le format Arc42 demandé, appliqué au projet formation-ecologie :

Dossier d'Architecture Technique (DAT)

Table des matières

- [Application Formation-Écologie](#)
- [1. Introduction et objectifs](#)
 - [Vue d'ensemble fonctionnelle](#)
 - [Schéma C4-L1 \(Vue Contexte\)](#)
 - [Objectifs de qualité orientés utilisateur](#)
- [2. Parties prenantes](#)
- [3. Contraintes](#)
 - [Contraintes techniques](#)
 - [Contraintes organisationnelles](#)
 - [Exigences de sécurité \(modèle D-I-C-T\)](#)
- [4. Contexte et périmètre](#)
 - [Partenaires fonctionnels](#)
 - [Interfaces techniques](#)
- [5. Stratégie de solution](#)
 - [Décisions architecturales majeures](#)
 - [Environnement technologique](#)
 - [Outils de la forge logicielle](#)
- [6. Vue en Briques \(Vue Conteneurs C4-L2\)](#)
 - [Description des conteneurs](#)
- [7. Vue Exécution](#)
 - [Scénario 1 : Recherche de formation avec critères multiples](#)
 - [Scénario 2 : Import quotidien des données CISIRH](#)
 - [Scénario 3 : Inscription d'un agent sans Self Agent \(hors ministère\)](#)
- [8. Vue Déploiement](#)
 - [Environnements](#)
 - [Infrastructure](#)
 - [Supervision](#)

- [Sauvegardes](#)
 - [9. Sujets transverses](#)
 - [Authentification](#)
 - [Journalisation](#)
 - [Monitoring](#)
 - [Gestion des erreurs](#)
 - [API et interfaces](#)
 - [10. Exigences de qualité](#)
 - [11. Risques et dettes techniques](#)
 - [12. Annexes](#)
 - [Glossaire](#)
 - [Décisions d'Architecture \(ADR\)](#)
-

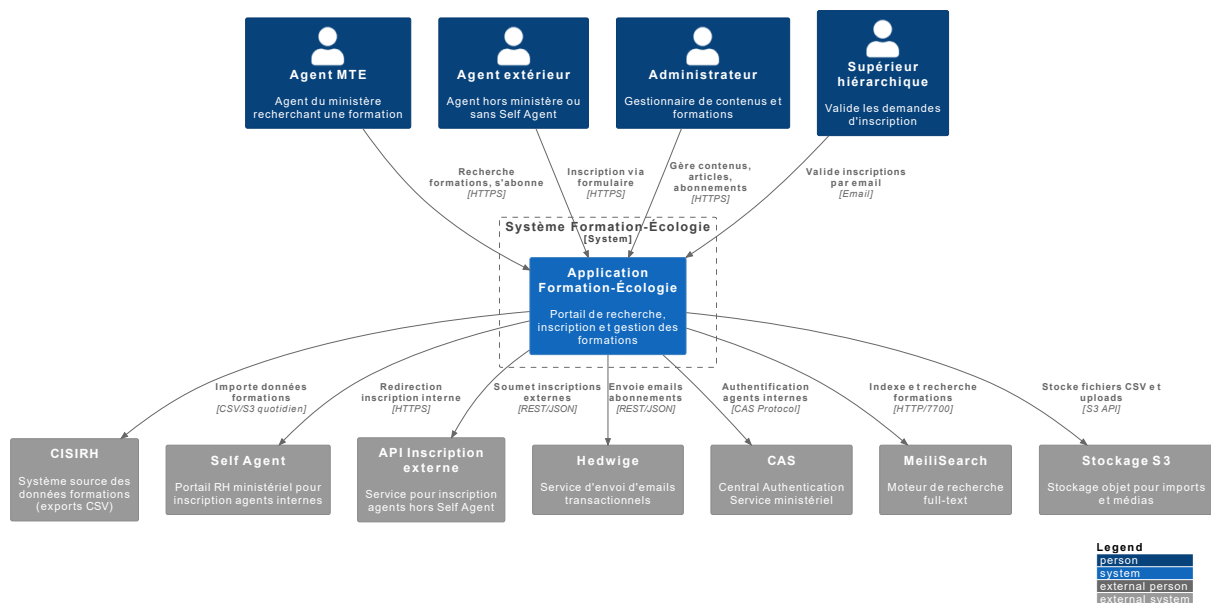
Application Formation-Écologie [↑](#)

1. Introduction et objectifs [↑](#)

Vue d'ensemble fonctionnelle [↑](#)

Formation-Écologie est une application web de gestion et diffusion de l'offre de formation professionnelle du pôle ministériel MTE (Ministère de la Transition Écologique). Elle permet aux agents de rechercher des formations, de s'y inscrire (directement ou via leur supérieur hiérarchique), et de s'abonner à des alertes email personnalisées. L'application intègre des données provenant du CISIRH (Centre Interministériel de Services Informatiques relatifs aux Ressources Humaines) et propose une interface d'administration pour la gestion des contenus éditoriaux.

Schéma C4-L1 (Vue Contexte) [↑](#)



Objectifs de qualité orientés utilisateur [↑](#)

- **Accessibilité** : Conformité RGAA 4.1.2 avec 82,35% des critères respectés pour garantir l'accès à tous les agents, y compris en situation de handicap
- **Performance** : Temps de réponse inférieur à 2 secondes pour les recherches de formations grâce à l'indexation MeiliSearch
- **Maintenabilité** : Architecture modulaire Django avec séparation claire des responsabilités (models, views, services) facilitant les évolutions
- **Sécurité** : Authentification CAS pour agents internes, chiffrement AES-256 des sauvegardes, conformité aux exigences RGS
- **Disponibilité** : Hébergement cloud redondant avec supervision 24/7 et procédures de backup automatisées

2. Parties prenantes [↑](#)

- **Direction des Ressources Humaines (DRH/MTE)**

- Attente : Offre de formation à jour, accessible et facilement consultable par tous les agents du pôle ministériel

- **SG/DNUM/DPNM/PNM3 (Équipe technique)**

- Attente : Hébergement sécurisé, supervision opérationnelle et maintenance évolutive de l'application

- **CISIRH (Fournisseur de données)**

- Attente : Fourniture fiable et quotidienne des données formations via exports CSV déposés sur S3

- **Agents du ministère (utilisateurs finaux)**

- Attente : Recherche intuitive des formations, inscription simplifiée et informations toujours à jour

- **Correspondants locaux de formation (CLF)**

- Attente : Gestion efficace des inscriptions des agents ne disposant pas d'accès au Self Agent

- **RSSI (Responsable de la Sécurité des Systèmes d'Information)**

- Attente : Conformité aux standards de sécurité, traçabilité des accès et protection des données personnelles

3. Contraintes [↑](#)

Contraintes techniques [↑](#)

- Stack technique imposée : Python 3.11+, Django 4.2 LTS, PostgreSQL 15+
- Hébergement obligatoire sur cloud interne ECO4 (OpenStack), tenant PNM3
- Conformité au Design System de l'État Français (DSFR) pour tous les composants UI
- Accessibilité : conformité partielle RGAA 4.1.2 (82,35% des critères)

Contraintes organisationnelles [↑](#)

- Authentification CAS ministérielle obligatoire pour les agents internes
- Traitement des données personnelles conforme au RGPD avec durée de conservation limitée
- Classification des données : niveau "diffusion restreinte"

Exigences de sécurité (modèle D-I-C-T) [↑](#)

- **Disponibilité** : 99,5% de disponibilité annuelle avec supervision continue via Prometheus/Grafana et alerting automatique
 - **Intégrité** : Protection contre l'altération des données via hash des fichiers importés et logs d'audit complets des actions administratives
 - **Confidentialité** : Chiffrement AES-256 des sauvegardes de base de données, accès restreints aux ressources sensibles, authentification forte
 - **Traçabilité** : Logs applicatifs détaillés, historique des imports CISIRH avec horodatage, traçabilité des envois d'emails d'abonnement
-

4. Contexte et périmètre [↑](#)

Partenaires fonctionnels [↑](#)

- **CISIRH** : Système source des données formations (stages, sessions, périodes) via exports CSV quotidiens
- **Self Agent** : Portail RH ministériel vers lequel sont redirigés les agents internes pour inscription
- **API Inscription externe** : Service tiers permettant la soumission des inscriptions pour agents sans Self Agent
- **Hedwige** : Service d'envoi d'emails transactionnels pour les notifications d'abonnement
- **CAS (Central Authentication Service)** : Service d'authentification unique pour les agents du ministère
- **MeiliSearch** : Moteur de recherche dédié pour l'indexation et la recherche full-text des formations
- **Stockage S3 (MinIO)** : Stockage objet pour les fichiers d'import CISIRH et les uploads de médias

Interfaces techniques [↑](#)

- **CISIRH → Application** : Import quotidien via fichiers CSV (STAGES.csv, SESSIONS.csv, PERIODES.csv) déposés sur bucket S3, téléchargement par script Python, parsing et insertion en base PostgreSQL
- **Application → Self Agent** : Redirection HTTP avec paramètres (code stage) vers URL du Self Agent
- **Application → API Inscription** : Requêtes HTTP POST en JSON vers endpoint externe pour soumission des formulaires d'inscription hors Self Agent

- **Application** → **Hedwige** : Requêtes HTTP POST authentifiées (OAuth2 client credentials) pour envoi d'emails HTML
 - **Application** → **CAS** : Protocole CAS 3.0 pour authentification, validation de tickets et récupération d'attributs utilisateur
 - **Application** → **MeiliSearch** : API HTTP REST (port 7700) pour indexation des documents et recherche avec filtres et tri
 - **Application** → **S3** : API S3 compatible pour upload/download de fichiers (CSV d'import, images d'articles)
-

5. Stratégie de solution [↑](#)

Décisions architecturales majeures [↑](#)

- Architecture monolithique Django choisie pour sa simplicité de maintenance avec une équipe réduite, tout en assurant une séparation logique claire via l'organisation en modules (models, views, services, forms)
- Indexation découplée avec MeiliSearch pour décharger la base PostgreSQL des requêtes de recherche complexes et garantir des performances constantes
- Traitement asynchrone des imports via Celery pour éviter le blocage du serveur web pendant le traitement des fichiers CSV volumineux
- Double mode d'authentification : CAS pour agents internes (flux standard) et formulaire libre pour agents externes (cas particuliers)

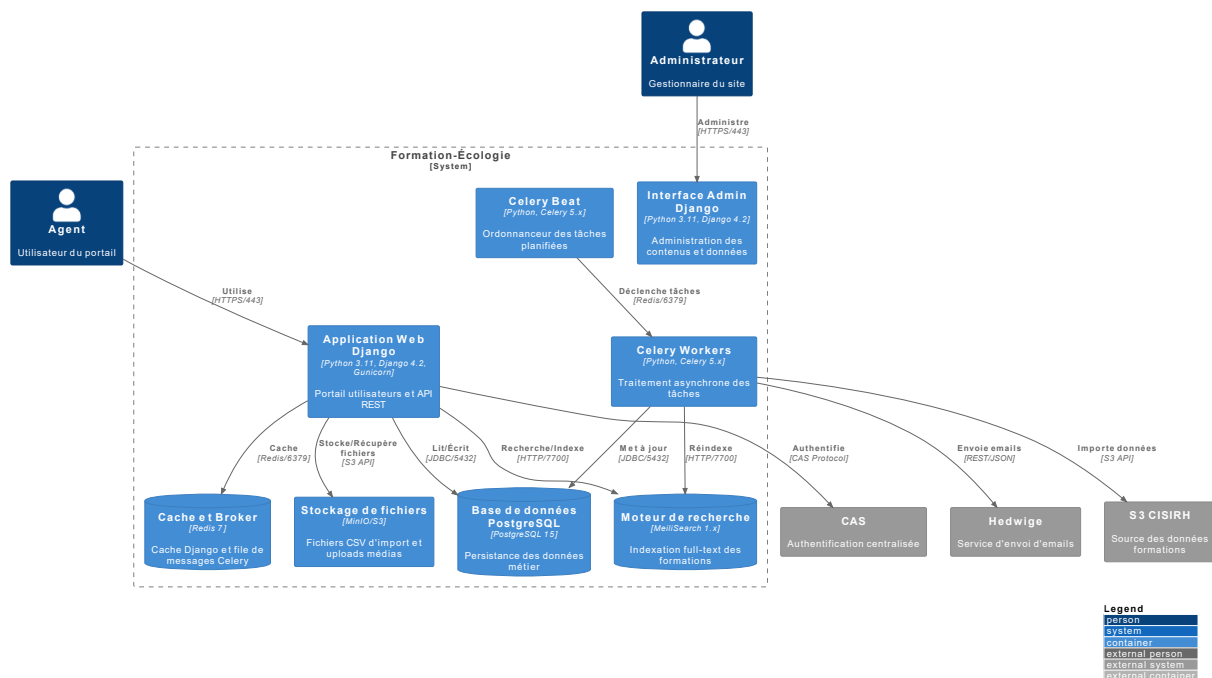
Environnement technologique [↑](#)

- **Langage** : Python 3.11+
- **Framework backend** : Django 4.2 LTS avec Django ORM
- **Base de données** : PostgreSQL 15
- **Moteur de recherche** : MeiliSearch 1.x
- **Cache et broker** : Redis 7
- **Tâches asynchrones** : Celery 5.x avec Celery Beat pour la planification
- **Frontend** : Django Templates, DSFR 1.14.2 (Système de Design de l'État), Leaflet 1.9.4 pour la cartographie
- **Conteneurisation** : Docker avec Docker Compose
- **Serveur d'application** : Gunicorn

Outils de la forge logicielle [↑](#)

- **Gestion de source** : GitLab (repository interne)
- **CI/CD** : GitLab CI avec pipeline de build d'image Docker via BuildKit
- **Gestion des dépendances** : Poetry avec fichier poetry.lock versionné
- **Linting** : Flake8 avec configuration personnalisée (max-line-length = 160)
- **Registry** : Registry GitLab interne pour le stockage des images Docker

6. Vue en Briques (Vue Conteneurs C4-L2) [↑](#)



Description des conteneurs [↑](#)

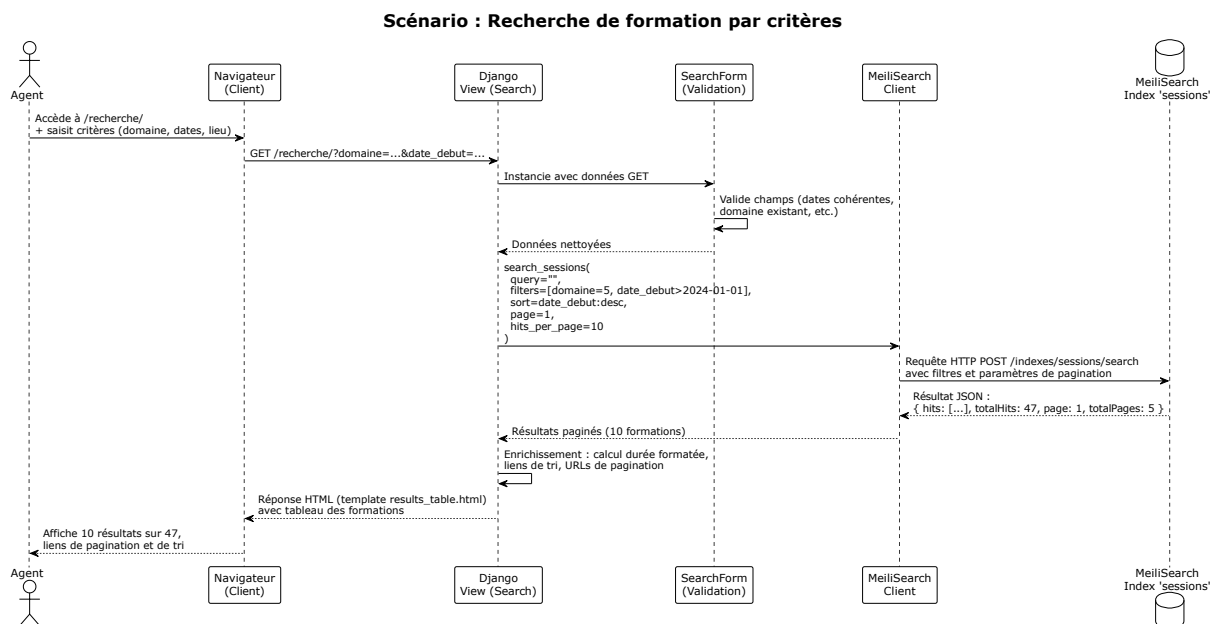
- **Application Web Django** : Cœur de l'application exposant le portail public (recherche de formations, inscription, abonnement) et les API REST internes. Sert les pages via Gunicorn avec workers synchrones.
- **Interface Admin Django** : Interface d'administration sécurisée pour la gestion des articles, bandeaux d'accueil, abonnements et suivi des imports. Partage le même code source mais avec routing et permissions spécifiques.
- **Celery Workers** : Processus workers exécutant les tâches asynchrones : import quotidien des données CISIRH, envoi des emails d'abonnement, réindexation MeiliSearch, nettoyage des fichiers temporaires.
- **Celery Beat** : Ordonnanceur déclenchant les tâches périodiques selon la crontab configurée (imports à 00:00, emails d'abonnement à fréquence définie).
- **Base de données PostgreSQL** : Stockage relationnel des données métier (stages, sessions, périodes, articles, abonnements, logs d'import) avec

réplication pour la haute disponibilité.

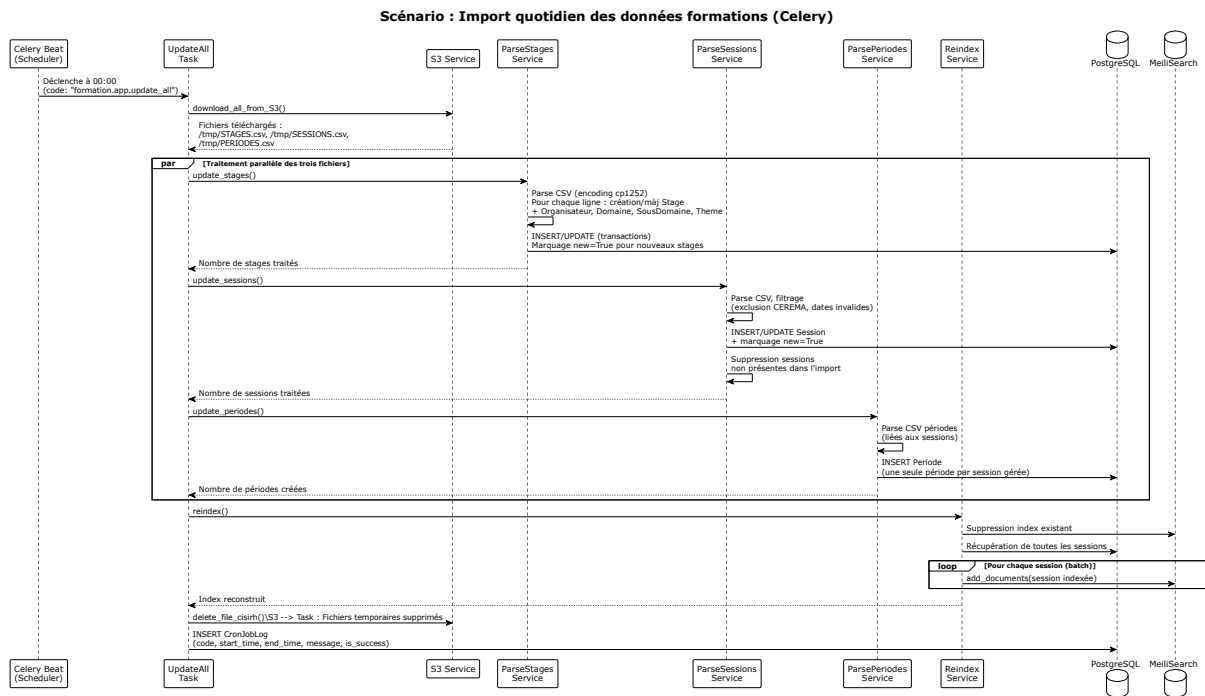
- **Moteur de recherche MeiliSearch** : Index dédié pour les formations (index "sessions" et "stages") permettant recherche full-text, filtrage multi-critères et tri rapide.
- **Cache et Broker Redis** : Utilisé comme cache de second niveau par Django et comme broker de messages pour la communication entre Celery Beat et les Workers.
- **Stockage de fichiers** : Service S3 compatible (MinIO) pour le stockage des fichiers CSV d'import en provenance du CISIRH, des images uploadées pour les articles, et des documents générés (exports PDF).

7. Vue Exécution [↑](#)

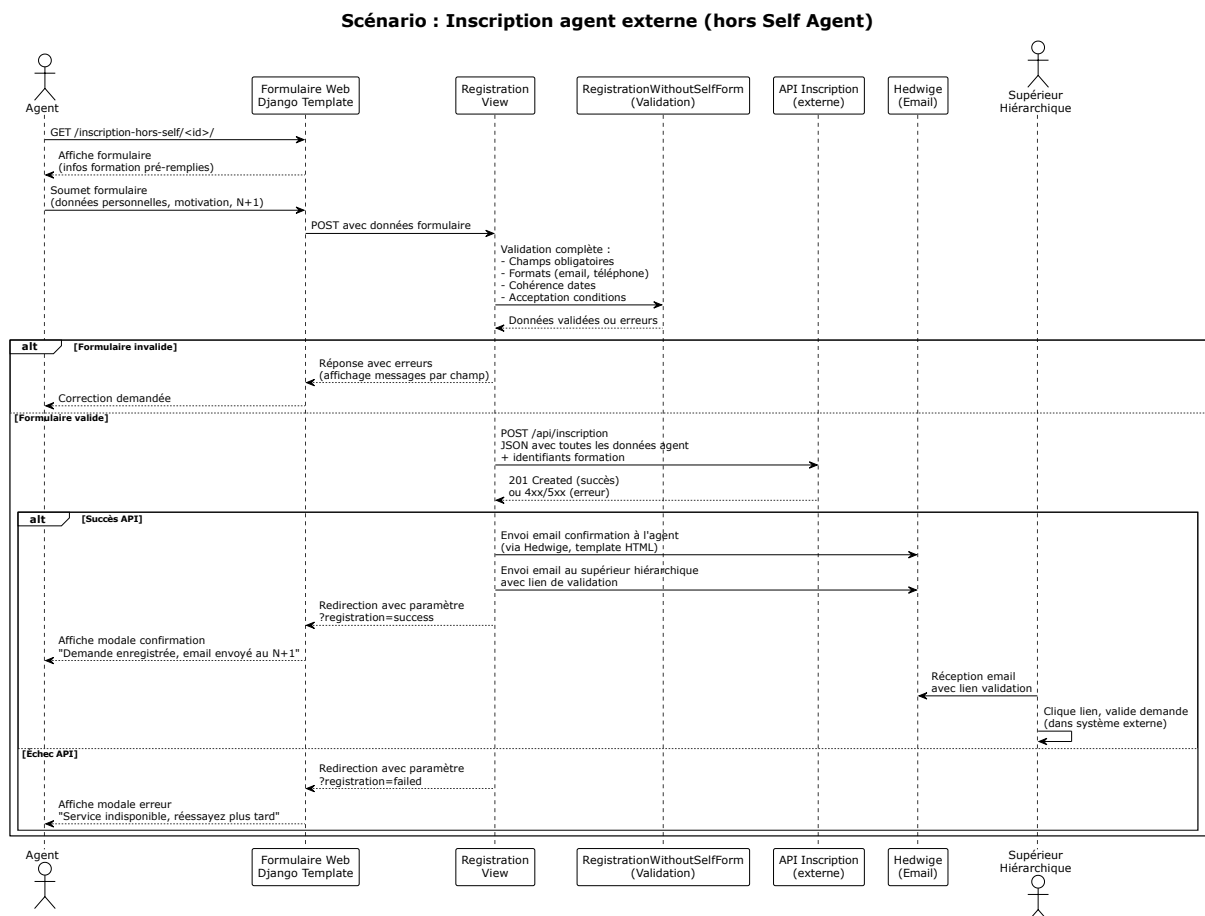
Scénario 1 : Recherche de formation avec critères multiples [↑](#)



Scénario 2 : Import quotidien des données CISIRH [↑](#)



Scénario 3 : Inscription d'un agent sans Self Agent (hors ministère) [↑](#)



8. Vue Déploiement [↑](#)

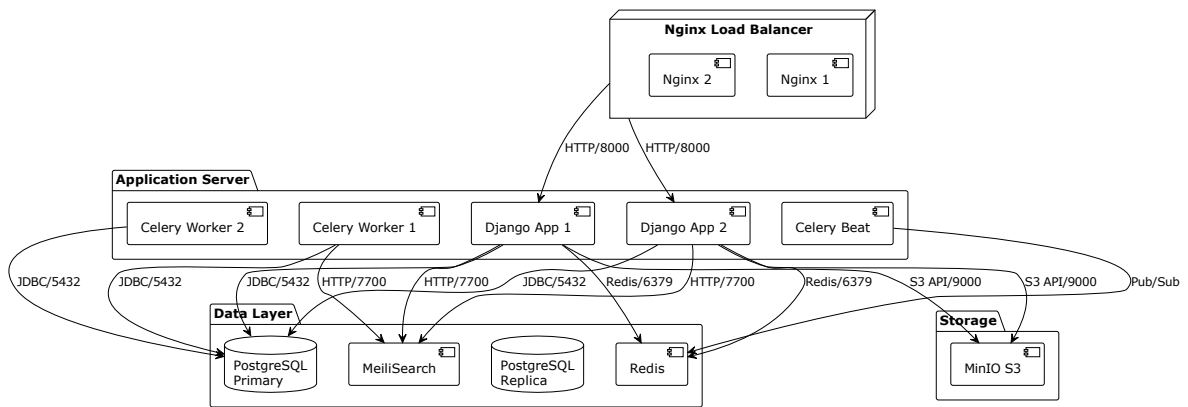
Environnements [↑](#)

Environnement	Hébergement	Serveurs	Réseau	Particularités
Développement	Local/Docker Desktop	1 (localhost)	Interne workstation	SQLite possible, DEBUG=True, rechargement automatique
Intégration	Cloud ECO4 (tenant dev)	2 (flavor small)	VLAN isolé 10.x.x.x/24	Données de test anonymisées, accès VPN restreint
Recette	Cloud ECO4 (tenant rec)	2 (flavor medium)	VLAN restreint 10.x.x.x/24	Données de pré-prod, tests RSSI et recette fonctionnelle
Production	Cloud ECO4 (tenant pnm3)	4+ (flavor medium+)	DMZ + réseau interne	Haute disponibilité, backup automatique, supervision renforcée

Infrastructure [↑](#)

Le produit est hébergé sur le cloud interne ECO4 basé sur Openstack, dans le tenant 'pnm3' du département.

Le reverse-proxy Nginx du schéma ci-dessous est en fait une paire de Nginx load-balancés en frontal des produits hébergés sur le tenant.



Supervision [↑](#)

Le produit est supervisé via le système standard du GTI pour ce faire :

- via Portainer pour la partie purement conteneurisée,
- via la stack Prometheus/Grafana/Loki/AlertManager,
- Le produit dispose également d'une supervision PSIN.

Sauvegardes [↑](#)

Les sauvegardes de la base de données sont assurées par des scripts standards du GTI permettant la création de dumps cryptés en AES-256 et déposés sur :

- le stockage objet B3 du IaaS ministériel,
- le stockage objet Outscale SecNumCloud (via la prestation qu'a le GTI sur le marché "Nuage Public"),
- le stockage objet standard de Google Cloud (via la prestation qu'a le GTI sur le marché "Nuage Public").

9. Sujets transverses [↑](#)

Authentification [↑](#)

- Mode CAS (production) : Authentification centralisée via le CAS ministériel pour tous les agents disposant d'un compte Active Directory
- Mode Local (développement) : Authentification Django standard avec gestion des utilisateurs en base locale
- Mode Anonyme : Certaines fonctionnalités (recherche, consultation) accessibles sans authentification
- Gestion des permissions : Django Groups et Permissions pour différencier administrateurs et utilisateurs standards

Journalisation [↑](#)

- Logs applicatifs : Format structuré (JSON) envoyé vers Loki pour centralisation et recherche
- Logs de sécurité : Tentatives d'accès, erreurs d'authentification, accès aux données sensibles
- Logs métier : Historique des imports CISIRH (table CronJobLog avec code, timestamps, message, statut succès/échec)
- Logs d'audit admin : Modifications des articles, création/suppression d'abonnements via l'interface Django

Monitoring [↑](#)

- Métriques applicatives : Nombre de recherches, taux de conversion inscription, volume d'abonnements
- Métriques techniques : Temps de réponse HTTP, taux d'erreur 5xx, saturation CPU/mémoire, file Celery
- Alertes : Indisponibilité service, erreurs d'import CISIRH, saturation disque, erreurs d'envoi d'emails

Gestion des erreurs [↑](#)

- Pages d'erreur personnalisées conformes DSFR pour les codes 403, 404, 500
- Fallback gracieux : Si MeiliSearch indisponible, affichage d'un message utilisateur explicite sans plantage
- Retry automatique : Tâches Celery configurées avec 3 tentatives et backoff exponentiel en cas d'échec

- Circuit breaker : Timeout sur les appels API externes (Hedwige, API Inscription) pour éviter blocage

API et interfaces [↑](#)

- API REST interne : Endpoints pour sous-domaines (/api/sous-domaines/) et départements par domaine (/api/departement-par-domaines/)
 - Flux RSS : Génération de flux RSS filtrables selon les critères de recherche de l'utilisateur
 - Webhooks : Aucun webhook externe utilisé actuellement
-

10. Exigences de qualité [↑](#)

- **Temps de réponse recherche < 2 secondes** : Validation par test de charge JMeter avec 100 requêtes de recherche simultanées par minute, mesure du 95e percentile des temps de réponse
 - **Disponibilité 99,5% annuelle** : Calcul sur base des 30 derniers jours glissants : (temps total en minutes - minutes d'indisponibilité déclarée) / temps total en minutes
 - **Conformité RGAA 82%+** : Audit annuel réalisé par société TEMESIS avec rapport de conformité détaillé et plan de remédiation
 - **Succès des imports quotidiens > 99%** : Mesure sur 30 jours du taux de succès des tâches Celery d'import CISIRH (code retour et validation données)
 - **Récupération backup complète < 4 heures** : Test semestriel de restauration complète de la base de données sur environnement isolé, chronométrage de la procédure
-

11. Risques et dettes techniques [↑](#)

- **Montée de version Django 4.2 → 5.x** : Impact moyen, probabilité élevée. Mesure : Planifier migration vers prochaine LTS en 2025, établir batterie de tests de non-régression automatisés, monitoring des dépréciations
- **Dépendance unique CISIRH comme source de données** : Impact élevé, probabilité moyenne. Mesure : Mise en place de mécanisme de cache étendu (7 jours), alerte immédiate en cas d'absence d'import quotidien, procédure de continuité avec données figées

- **MeiliSearch en instance unique (non clusterisé)** : Impact moyen, probabilité faible. Mesure : Backup quotidien de l'index via script automatisé, documentation de la procédure de reconstruction complète de l'index (15-30 minutes)
 - **Dette technique : fonctions de parsing CSV longues et complexes** : Impact moyen, probabilité élevée. Mesure : Refactoring progressif par extraction de méthodes, augmentation de la couverture de tests unitaires sur le module services
 - **Stockage des secrets en variables d'environnement** : Impact faible, probabilité moyenne. Mesure : Évaluation de la migration vers Vault ou équivalent fourni par le GTI, rotation régulière des clés API
-

12. Annexes [↑](#)

Glossaire [↑](#)

- **CISIRH** : Centre Interministériel de Services Informatiques relatifs aux Ressources Humaines, fournisseur des données formations
- **Self Agent** : Portail RH ministériel permettant aux agents de gérer leurs demandes de formation
- **DSFR** : Design System de l'État Français, bibliothèque de composants UI accessibles et conformes à l'identité visuelle de l'État
- **MeiliSearch** : Moteur de recherche open source léger et performant, alternative à Elasticsearch
- **Celery** : Framework de traitement distribué de tâches asynchrones pour Python
- **CAS** : Central Authentication Service, protocole de Single Sign-On (SSO) utilisé par le ministère
- **Hedwige** : Service d'envoi d'emails transactionnels de la DNUM
- **CVRH/CMVRH** : Centre de Valorisation des Ressources Humaines, organisateurs de formations
- **DREAL/DDT/DIR** : Directions régionales et départementales, organisateurs de formations

Décisions d'Architecture (ADR) [↑](#)

- **ADR-001 (2024-01)** : Choix de MeiliSearch comme moteur de recherche plutôt qu'Elasticsearch pour sa simplicité de déploiement et sa consommation

mémoire réduite. Statut : Accepté.

- **ADR-002 (2024-02)** : Maintien d'une authentification double mode (CAS + formulaire libre) pour prendre en compte les agents externes et agents de terrain sans accès Self Agent. Statut : Accepté.
- **ADR-003 (2024-03)** : Utilisation de Celery pour le traitement asynchrone des imports CISIRH afin de ne pas bloquer le serveur web pendant le parsing de fichiers CSV volumineux. Statut : Accepté.
- **ADR-004 (2024-06)** : Obligation d'utilisation du DSFR pour tous les composants UI afin de garantir l'accessibilité et l'homogénéité visuelle avec les autres applications de l'État. Statut : Accepté.
- **ADR-005 (2024-11)** : Stockage des fichiers d'import sur S3 plutôt que sur volume local pour faciliter le partage de données avec le CISIRH et permettre la scalabilité horizontale. Statut : Accepté.

[↩ Retour au sommaire](#)

Document généré le 28 février 2026 - Version 1.0